Bursa Map

Proje Yapısı Rehberi

Mimari, veri akışı, modüller, değerlendirme ve öneriler

Hazırlanma tarihi: 12 Ekim 2026 · Kod tabanı: ~10.000 satır TypeScript/JavaScript · 160 birim testi

İçindekiler

1. Proje özeti 3

2. Mimari genel bakış 4

3. Klasör yapısı 5

4. Veri katmanı 6

5. İstemci (tarayıcı) tarafı 7

6. Sunucu tarafı: yol tarifi 8

7. Testler, araçlar ve çalışma komutları 9

8. Değerlendirme: artılar ve eksiler 10

9. Bu incelemede yapılan düzenlemeler 11

10. Öneriler (öncelik sırasıyla) 11

11. Yeni geliştirici için okuma sırası 12

1. Proje özeti

Bursa Map, Bursa için mobil öncelikli bir toplu taşıma haritasıdır (PWA). Metro (BursaRay M1/M2), tramvay (T1–T3) ve 409 otobüs hattını tek haritada gösterir; durak, hat, yer ve adres arar; kullanıcının kayıtlı yerlerini tutar ve otobüs + raylı sistem üzerinde **sunucu tarafında RAPTOR algoritmasıyla** yol tarifi üretir. Yürüme bacakları, projenin kendi OSM tabanlı yaya yolu ağı üzerinde Dijkstra/A* ile hesaplanır.

Temel özellikler

- Harita: OpenFreeMap vektör altlık + kendi katmanlar (raylı hatlar, otobüs durakları, öne çıkan yerler, çevredeki yerler), 2D/3D görünüm.
- Arama: Türkçe duyarlı, aksan ve harf hatası affeden arama (ör. *sehrekustu* → Şehreküstü); ardından Photon ile adres sonuçları.
- Hatlar paneli: tüm otobüs hatları, filtreleme, seçilen hattın güzergâhı ve durakları (hat başına ayrı dosya indirilir).
- Yol tarifi: Nereden/Nereye, "Konumum", kalkış saati seçimi; seçenekler en hızlı / en ucuz / en az aktarma / en az yürüme olarak etiketlenir; tahmini BursaKart ücreti.
- Kayıtlı: yerler ve hatlar tarayıcıda (localStorage) kararlı kimliklerle saklanır.
- Çevrimdışı: service worker harita karolarını ve açılan hat güzergâhlarını önbelleğe alır.

Teknoloji yığını

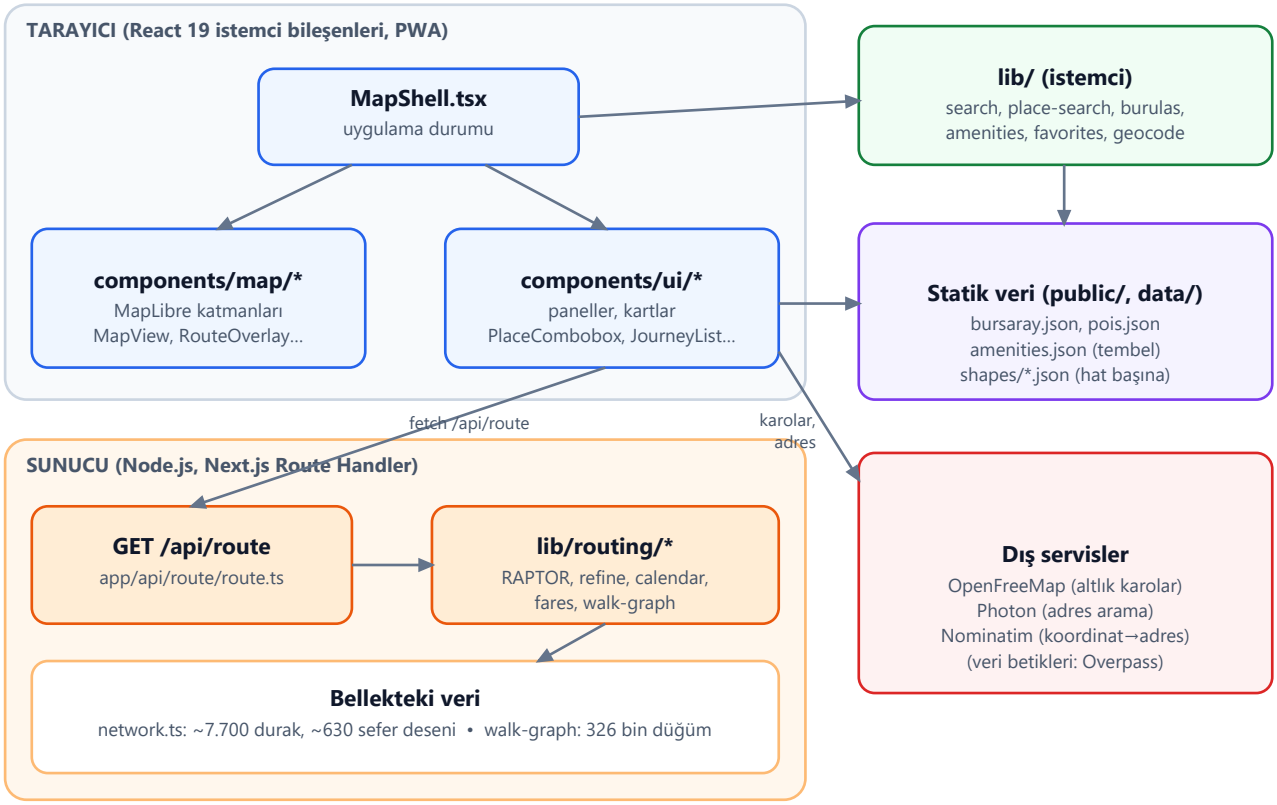
Katman	Teknoloji	Not
Çatı	Next.js 16.4 (App Router), React 19.3	Tek sayfa (app/page.tsx) + tek API ucu (app/api/route)
Dil	TypeScript 6 (strict)	Yol takma adı `@/*` → proje kökü
Harita	MapLibre GL 6 + react-map-gl 8	OpenFreeMap "Liberty" stili, API anahtarı yok
Stil	Tailwind CSS 4	app/globals.css içinde birkaç özel sınıf (glass, route-endpoint)
Durum	React state + Zustand 5	Zustand sadece favoriler için (persist → localStorage)
PWA	@ducanh2912/next-pwa (Workbox)	Webpack gerektirdiği için build `--webpack` ile
Test	Vitest 5	Node ortamında saf iş mantığı testleri
Kalite	ESLint 9 (eslint-config-next)	`npm run lint`, `npm run typecheck`

Veri hacmi (Ekim 2026)

Veri	Miktar	Kaynak
Otobüs hatları / durakları	409 hat, 7.714 durak (7.549'u hatlı)	BURULAŞ / BursaKart
Sefer kalkışları	127.925 kalkış (gün gün)	BURULAŞ
Raylı sistem	M1, M2, T1, T2, T3 (79 GeoJSON öğesi)	OpenStreetMap
Öne çıkan yerler	32 nokta (kampüs, AVM, hastane, otogar…)	OpenStreetMap
Çevredeki yerler	5.543 nokta, 9 kategori	OSM + All The Places + Nominatim
Yaya yolu ağı	325.776 düğüm, 364.497 kenar (~5 MB gzip)	OpenStreetMap

2. Mimari genel bakış

Uygulama iki parçadan oluşur: tarayıcıda çalışan tek sayfalık harita arayüzü ve sunucuda çalışan yol tarifi ucu. Ağır hesaplama (RAPTOR + yaya ağı araması, onlarca MB bellek) sunucuda kalır; tarayıcı yalnızca ihtiyaç duyduğu veriyi tembel olarak yükler.



Tasarım kararları

- **Tek durum kabuğu:** `components/MapShell.tsx` sekme, katmanlar, arama metni, seçili yer, rota uçları ve kalkış saatini tutar; alt bileşenler sadece prop ve geri çağırma alır (tek yönlü veri akışı).
- **Saf mantık lib/ altında:** arama, takvim, ücret, RAPTOR, yaya ağı gibi tüm iş mantığı React'ten bağımsız fonksiyonlardır; bu yüzden testler jsdom olmadan Node'da çalışır.
- **Tembel yükleme:** BURULAŞ verisi (~1 MB) tarayıcı boşta kalınca, çevredeki yerler zoom ≥ 13 veya arama odağında, hat güzergâhı ise sadece seçilince indirilir.
- **Uzamsal ızgaralar:** yakın durak, çevredeki yer ve yol parçası aramaları sabit boyutlu hücre ızgaralarıyla yapılır (tam tarama yok).
- **Sunucu belleğinde önbellek:** `getNetwork()` ve `getWalkGraph()` ilk çağrıda kurulur, süreç yaşadıkça yeniden kullanılır.
- **Harici servis sayısı az:** yol tarifi hiçbir dış servise gitmez; sadece altlık karoları (OpenFreeMap) ve adres aramaları (Photon/Nominatim) dışarıya istek atar.

3. Klasör yapısı

```
bursa-map/
├── app/                        Next.js App Router
│   ├── layout.tsx              <html lang="tr">, metadata, viewport
│   ├── page.tsx                tek sayfa → <MapShell/>
│   ├── manifest.ts             PWA manifest
│   ├── globals.css             Tailwind 4 + özel sınıflar
│   └── api/route/route.ts      GET /api/route  (yol tarifi, sunucu)
├── components/
│   ├── MapShell.tsx            uygulama kabuğu ve durum
│   ├── map/                    MapLibre: Map (dynamic, ssr:false), MapView,
│   │                           TransitLayer, BusStopsLayer, BusLineLayer,
│   │                           PoiLayer, AmenityLayer, RouteOverlay
│   └── ui/                     SearchBar, PlaceCombobox, ResultCards,
│                               RoutePanel, JourneyList, LinesPanel, SavedPanel,
│                               SheetPanel, BottomNav, LayerControl,
│                               FavoriteButton, PlaceVisuals
├── data/                       işlenmiş veri + tipler
│   ├── types.ts                Place, LayerId, BusLine, GeoJSON tipleri
│   ├── layers.ts places.ts pois.ts transit-network.ts
│   ├── bursaray.json pois.json amenities.json
│   ├── burulas/                BURULAŞ otobüs veri seti (json + xlsx)
│   ├── walk/                   yaya ağı (bursa-walk.bin.gz)
│   └── raw/                    ham indirmeler (git dışı)
├── lib/                        iş mantığı
│   ├── search.ts place-search.ts geocode.ts reverse-geocode.ts
│   ├── burulas.ts amenities.ts favorites.ts geolocation.ts
│   ├── map-config.ts map-features.ts geo.ts use-*.ts
│   └── routing/                network, raptor, refine, walk-graph,
│                               walking, calendar, fares, time, types, client
├── scripts/                    veri üretim betikleri (Node .mjs)
├── tests/                      Vitest (14 dosya, 160 test)
├── public/                     ikonlar; build'de üretilen sw.js, vendor/, data/
└── next.config.mjs  vercel.json  vitest.config.mts  eslint.config.mjs
```

Dosya adlandırma kuralları: React bileşenleri PascalCase (`RoutePanel.tsx`), modüller kebab-case (`place-search.ts`), hook dosyaları `use-` önekiyle. Kod yorumları ve kullanıcı metinleri Türkçedir.

4. Veri katmanı

Uygulama çalışırken hiçbir veritabanı kullanmaz; tüm veri betiklerle üretilip depoya JSON/ikili dosya olarak girer. Aşağıdaki tablo her dosyanın kim tarafından üretildiğini ve kim tarafından kullanıldığını gösterir.

Dosya	Üreten	Kullanan	Yükleme
data/bursaray.json	fetch-bursaray, patch-rail-gaps	transit-network.ts → TransitLayer, places	Ana paket
data/pois.json	fetch-pois	pois.ts → PoiLayer, arama	Ana paket
data/amenities.json	fetch-amenities	lib/amenities.ts → AmenityLayer, arama	Tembel (import())
data/burulas/lines.json, stations.json	build-bus-dataset	lib/burulas.ts (istemci), network.ts (sunucu)	Tembel / sunucu
data/burulas/line-stops.json, schedules.json	build-bus-dataset	lib/routing/network.ts	Sadece sunucu
data/burulas/line-shapes.json	build-bus-dataset	split-shapes → public/data/burulas/shapes/	Hat başına fetch
data/walk/bursa-walk.bin.gz	fetch-walk-graph	lib/routing/walk-graph.ts	Sunucu, fs ile
data/burulas/*.xlsx	build-bus-dataset	İnsan okuması için (uygulama kullanmaz)	—

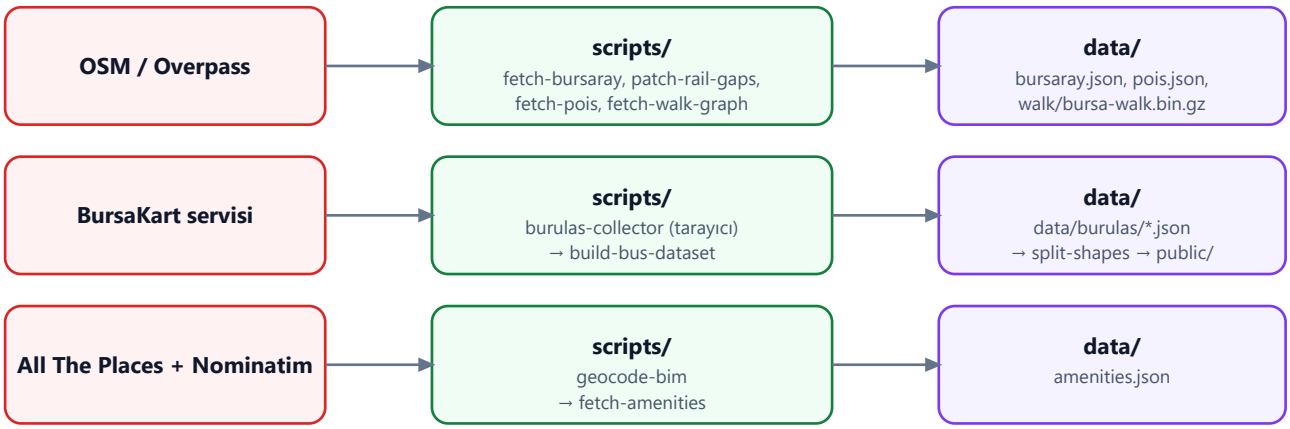
Yer (Place) modeli

Aranabilen ve seçilebilen her şey `data/types.ts` içindeki **Place** birleşim tipidir; `kind` alanı türü belirler. Konumlar uygulama içinde **[enlem, boylam]**, GeoJSON/MapLibre'de **[boylam, enlem]** sırasındadır; çeviri `toLngLat()` ile yapılır.

kind	Örnek kimlik	Açıklama
poi	bursa-terminal	Öne çıkan yer (pois.json)
station	metro-bursaspor-acemler, bus-stop-3467	Raylı istasyon ya da otobüs durağı
line	bus-line-22C	Otobüs hattı (BusLine nesnesiyle)
amenity	amenity-n123456	Market, okul, eczane, cami, park, banka/ATM, PTT
feature	feature-..., pin-..., address-..., my-location	Altlıktan dokunulan yer, iğne, adres sonucu, cihaz konumu

Kimlikler kararlıdır: favoriler bu kimliklerle saklanır ve veri yenilendiğinde güncel bilgiyle yeniden eşleştirilir.

Veri üretim hattı



BURULAŞ verisi yalnızca bursakart.com.tr kaynaklı isteklere cevap veren bir servisten toplandığından toplama tarayıcı konsolunda `scripts/burulas-collector.browser.js` ile yapılır. Ardından `build-bus-dataset.mjs` ham dosyayı uygulama formatına çevirir. `split-shapes.mjs` her `npm run dev/build` öncesinde otomatik çalışır.

5. İstemci (tarayıcı) tarafı

Bileşen ağacı

```
app/page.tsx
└── MapShell                        durum: tab, visibleLayers, query, selectedPlace,
    │                               route{from,to}, departAt, journeyChoice, notice
    ├── BursaMap (Map.tsx, dynamic ssr:false)
    │   └── MapView                 MapLibre haritası, tıklama yönlendirme, 3D düğmesi
    │       ├── BusStopsLayer       ~7.500 durak (zoom ≥ 14,5)
    │       ├── AmenityLayer        çevredeki yerler (merkezde 1 km / ekran)
    │       ├── TransitLayer ×2     tramvay, metro (hat + istasyon + etiket)
    │       ├── PoiLayer            öne çıkan yerler
    │       ├── BusLineLayer        seçili hat (çizgi + 3D şerit + duraklar)
    │       ├── SelectedMarker / FlyToSelected
    │       └── RouteOverlay        A/B işaretleri, seçili rotanın bacakları
    ├── SearchBar → PlaceCombobox   yerel arama + Photon adres sonuçları
    ├── ResultCards                 sonuç kartları / seçili yerin bilgi kartı
    ├── RoutePanel                  Nereden/Nereye, saat, JourneyList
    ├── LinesPanel, SavedPanel      SheetPanel tabanlı alt paneller
    ├── LayerControl                katman aç/kapa
    └── BottomNav                   Harita · Rota · Hatlar · Kayıtlı
```

Önemli istemci modülleri

Modül	Sorumluluk
lib/search.ts	Türkçe katlama (*foldTurkish*), kelime-başı önek indeksi (1–2 harf), puanlama (tam=0, baştan=1, kelime başı=2), kesin sonuç yoksa Damerau-Levenshtein ile harf hatası toleransı (4–6 harf: 1, 7+: 2 hata).
lib/place-search.ts	Uygulama genelinde tek arama indeksi; otobüs ve çevredeki yerler yüklendikçe yeniden kurulur, `useSyncExternalStore` ile abonelere bildirilir. Öncelik (boost) ve haritaya yakınlık sıralaması.
lib/burulas.ts	Otobüs verisinin tembel yüklenmesi, durak adı biçimlendirme (Türkçe büyük/küçük harf), hat güzergâhı dosyası önbelleği.
lib/amenities.ts	Sıkıştırılmış satır formatını Place'e çevirir; ~1 km ızgarayla *near()* ve *inBounds()* sorguları.
lib/geocode.ts / reverse-geocode.ts	Photon (adres → koordinat; Türkçe kısaltma açma, Bursa kutusu) ve Nominatim (koordinat → adres; 1,1 sn kısıtlama kuyruğu). Uç noktalar `NEXT_PUBLIC_*` ortam değişkenleriyle değiştirilebilir.
lib/favorites.ts	Zustand + persist; SSR uyuşmazlığı olmasın diye *skipHydration* ve ilk render sonrası yükleme.
lib/routing/client.ts	`useJourneys`: URL'yi anahtar yaparak /api/route çağırır, AbortController ile eski isteği iptal eder.
lib/geolocation.ts	Konum izni, güvenli bağlam (HTTPS) kontrolü, Bursa dışı konum reddi, Türkçe hata mesajları.

Durum yönetimi deseni

Asenkron hook'lar (`useJourneys`, `useAddressSearch`, `useReverseGeocode`) sonucu **{anahtar, durum}** çifti olarak saklar; anahtar değişince "yükleniyor" durumu efekt içinde setState ile değil anahtar uyuşmazlığından türetilir. Böylece yarış durumları ve ekstra render turları önlenir. RoutePanel'deki metin kutuları ise prop değişince render sırasında güncellenir (React'in *you might not need an effect* önerisi).

PWA ve önbellek

- Hat güzergâhları: StaleWhileRevalidate (450 dosya, 30 gün).
- Altlık karoları, fontlar, sprite'lar: CacheFirst (3.000 kayıt, 30 gün). Stil JSON'u NetworkFirst.
- Service worker sadece production'da; geliştirmede devre dışı (Turbopack ile çakışmasın diye PWA sarmalayıcısı da yalnızca production'da).

6. Sunucu tarafı: yol tarifi



Toplu taşıma ağı ve yaya ağı ilk istekte kurulur, süreç boyunca bellekte kalır (sonraki aramalar ms düzeyinde).

API sözleşmesi

```
GET /api/route?from=40.2123,29.0145&to=40.2659,29.0532
              [&fromName=…&toName=…][&at=2026-10-08T08:30:00+03:00]

200 → { journeys: Journey[], serviceDay: 1..7, queryTime: saniye, notes: string[] }
400 → { error: "…" }   (Bursa kutusu dışında / hatalı tarih)

Journey = { departure, arrival, durationMin, transfers, walkMeters,
            distanceMeters, fare, tags[], legs: (WalkLeg | RideLeg)[] }
```

Modüller

Dosya	Ne yapar
routing/network.ts	Durakları, sefer desenlerini (hat + yön; durak sırası, ilk duraktan süre ofsetleri, gün gün kalkışlar) ve 400 m içindeki yürüyerek aktarmaları kurar. Yakın durak araması ~550 m'lik ızgarayla.
routing/raptor.ts	RAPTOR (Delling vd. 2012). k. tur "en fazla k araçla" en erken varışları bulur (≤5 araç). Aktarmada 3 dk pay, gece yarısı öncesi/sonrası seferler ±24 saat kaydırılarak eklenir. *findAlternatives* bulunan hatları tek tek yasaklayıp yeniden arar (≤16 arama). *rankJourneys* varış + aktarma başına 5 dk cezayla sıralar, en hızlıdan 1 saatten geç olanları eler, ölçüt etiketlerini dağıtır.
routing/walk-graph.ts	CSR formatında sıkıştırılmış yaya grafı (typed array'ler). Noktayı en yakın yol *parçasına* yapıştırır (≤400 m), çok kaynaklı Dijkstra ve hedefe A*; arama tamponları "nesil" damgasıyla yeniden kullanılır.
routing/refine.ts	Aramadan önce başlangıç/varış çevresindeki duraklara gerçek yürüme mesafesi; aramadan sonra yürüme bacaklarına sokak güzergâhı, saatlerin yeniden hesaplanması ve yetişilemeyen aktarmaların elenmesi.
routing/calendar.ts	Europe/Istanbul saatiyle tarife günü; resmi ve dini bayramlarda Pazar tarifesi. Dini bayramlar **2027 sonuna kadar** tanımlı.
routing/fares.ts	BursaKart tam bilet tarifesi (1 Ekim 2026): metro 50 TL, tramvay 40,75 TL, kısa hat otobüs 47 TL, ilçe hatları ayrı; 90 dk içinde ≤3 aktarma 21,50 TL.
routing/time.ts	Saat ayrıştırma/biçimlendirme, Haversine, durak ofseti tahmini (otobüs 20, tramvay 16, metro 35 km/s; yol katsayısı 1,2).

Bilinen varsayımlar

- Veride yalnızca ilk duraktan kalkış saatleri var; ara durak saatleri sabit ortalama hızla **tahmin** edilir (trafik yok sayılır). Arayüz bunu "~" ile ve notlarla belirtir.
- Yaya ağı durakların ~%98'ini kapsayan dikdörtgende; dışındaki uzak ilçelerde yürüme kuş uçuşu × 1,25 tahminidir.
- Ağ ilk istekte kurulur (~0,5 sn soğuk başlangıç), sonraki aramalar milisaniyeler sürer; yanıt `Server-Timing` başlığında süreyi bildirir.

7. Testler, araçlar ve çalışma komutları

Test dosyası	Kapsam
raptor.test.ts, street-plan.test.ts, walking.test.ts	RAPTOR seçenekleri, sokak ağıyla tam akış, yaya grafı (sentetik graf ile)
api-route.test.ts	Route handler: parametre doğrulama, notlar, hata cevapları
calendar.test.ts, routing-time.test.ts, fares.test.ts	Tarife günü/bayramlar, saat ve süre hesapları, ücret ve aktarma indirimi
search.test.ts, place-search.test.ts	Türkçe katlama, puanlama, harf hatası toleransı, öncelikler
geocode.test.ts, reverse-geocode.test.ts, geolocation.test.ts	Adres servisleri (fetch taklidi), konum hataları
amenities.test.ts, data-ids.test.ts	Izgara sorguları; veri dosyalarındaki kimliklerin kararlılığı ve tekilliği

Testler `TZ=UTC` ve `WALK_GRAPH=off` ile çalışır: sonuçlar makineden bağımsızdır ve gerçek 5 MB'lık yaya ağı yüklenmez.

Komutlar

Komut	Açıklama
npm install	Bağımlılıklar; postinstall MapLibre worker'ını public/vendor/ altına kopyalar
npm run dev	Geliştirme sunucusu (Turbopack), önce split-shapes
npm run build / npm start	Production derleme (webpack, PWA) / çalıştırma
npm test · npm run test:watch	Vitest
npm run lint · npm run typecheck	ESLint · tsc --noEmit
npm run data	OSM verilerini yeniler (pois, amenities, bursaray, rail-gaps)
npm run data:walk	Yaya ağını yeniden üretir (birkaç dakika, 2 GB bellek)
node scripts/geocode-bim.mjs	BİM adreslerini koordinata çevirir (~15 dk, Nominatim 1 istek/sn)

Ortam değişkenleri

Değişken	Varsayılan	Not
NEXT_PUBLIC_REVERSE_GEOCODE_URL	nominatim.openstreetmap.org	Yayında kendi/ücretli sunucu zorunlu (OSM politikası)
NEXT_PUBLIC_GEOCODE_URL	photon.komoot.io	Adil kullanım sınırı; yoğun trafikte kendi Photon sunucusu
OVERPASS_URL, NOMINATIM_URL	resmi sunucular	Sadece veri betikleri için

Dağıtım

Yol tarifi sunucuda çalıştığından Node.js destekli bir ortam gerekir (Vercel önerilir). Sunucu bölgesi `vercel.json` içinde **fra1** (Frankfurt). Yaya ağı dosyası fs ile okunduğu için `next.config.mjs → outputFileTracingIncludes` ile fonksiyon paketine elle eklenir.

8. Değerlendirme: artılar ve eksiler

Artılar

- **Sağlam mühendislik temeli:** TypeScript strict, lint ve tip kontrolü temiz, production derlemesi uyarısız; 160 birim testi 3 saniyede geçiyor.
- **Gerçek bir rota motoru:** RAPTOR + alternatif arama + kendi yaya ağı; yol tarifi için dış servise bağımlılık ve kota yok, arama başına milisaniyeler.
- **Performans bilinci:** tembel yükleme, hat başına güzergâh dosyaları, ızgara indeksleri, typed array'ler, yeniden kullanılan arama tamponları, akışla (stream) bölünen büyük JSON.
- **Türkçe'ye özgü kullanıcı deneyimi:** I/ı-İ/i doğru katlama, harf hatası toleransı, adres kısaltmalarının açılması, Türkçe durak adı biçimlendirme.
- **Erişilebilirlik:** ARIA combobox deseni, klavye gezinmesi, *inert* paneller, *prefers-reduced-motion* desteği, ekran okuyucu metinleri.
- **Mobil ve çevrimdışı:** PWA, safe-area desteği, iOS zoom önlemi, karo önbelleği.
- **Belgelenmiş kod:** neredeyse her modülün başında "neden" açıklayan Türkçe yorumlar; README kurulum, veri yenileme ve dağıtımı anlatıyor.
- **Yeniden üretilebilir veri:** tüm veri betiklerle üretiliyor; Overpass için tekrar deneme, Nominatim için hız sınırı uygulanıyor.

Eksiler ve riskler

- **Yasal risk (en kritik):** BURULAŞ verisi herkese açık sunulmayan bir servisten toplanıyor; yayından önce kullanım izni alınmalı (README'de not var).
- **Tahmini saatler:** ara durak saatleri sabit hızla tahmin; gerçek zamanlı araç konumu ya da trafik yok. Uzun hatların sonunda sapma büyüyebilir.
- **Elle bakım gerektiren tablolar:** dini bayram tarihleri 2027 sonuna, ücret tarifesi 1 Ekim 2026'ya sabit; güncellenmezse sessizce eskir (API not ekliyor).
- **Korumasız API ucu:** /api/route'ta hız sınırlama yok; her istek ≤16 RAPTOR araması yapıyor — kötüye kullanımda sunucu maliyeti artabilir.
- **Ücretsiz harici servis varsayılanları:** Nominatim/Photon genel sunucuları yayın trafiği için uygun değil; ortam değişkeni ayarlanmazsa engellenme riski.
- **Test kapsamı tek katmanlı:** sadece lib/ testleri var; bileşen testi ve uçtan uca (Playwright) test yok, CI tanımlı değil.
- **Büyük bileşen durumu:** MapShell ~300 satır ve çok sayıda useState içeriyor; özellik eklendikçe bakım zorlaşabilir.
- **Depo boyutu:** büyük JSON'lar, xlsx ve 5 MB ikili dosya git geçmişine giriyor; her veri yenilemesi depoyu büyütür.
- **Derleme kısıtı:** next-pwa webpack gerektirdiği için production'da Turbopack kullanılamıyor.

9. Bu incelemede yapılan düzenlemeler

İnceleme öncesinde tip kontrolü, lint ve testler zaten temizdi; production derlemesi bir kullanımdan kalkma uyarısı veriyordu. Yapılan değişiklikler davranışı korur; sonrasında tip kontrolü, lint, 160 test ve production derlemesi yeniden çalıştırılıp doğrulandı.

Dosya	Değişiklik	Neden
app/api/route/route.ts, vercel.json (yeni)	`preferredRegion = "fra1"` kaldırıldı; bölge vercel.json'a taşındı	Next 16.4'te kullanımdan kalktı, derleme uyarı veriyordu
vitest.config.ts → vitest.config.mts	Uzantı değişti	Vite "ESM sözdizimi CommonJS olarak yüklendi" uyarısı (gelecek sürümde hata)
lib/routing/raptor.ts, refine.ts	*estimatedWalks* tek yerde tanımlandı, refine.ts onu içe aktarıyor	Aynı fonksiyonun iki kopyası vardı
lib/geo.ts	*haversineMeters* routing/time.ts'teki uygulamayı kullanıyor	Kod tekrarı
components/ui/PlaceCombobox.tsx	Aşağı ok tuşu seçim indeksini 0'ın altına düşürmüyor	Liste boşken indeks −1 oluyordu
data/types.ts	Kullanılmayan 4 tip koruyucu (isTramLine, isBusStop…) silindi	Ölü kod
Map.tsx, geo.ts, map-config.ts, types.ts	Eski "Leaflet" yorumları güncellendi	Proje MapLibre'ye geçmiş, yorumlar yanıltıcıydı
next.config.mjs, MapView.tsx, README.md	Güzergâh boyutu yorumu (~12 MB), JSX biçimi, dağıtım notu	Doğruluk ve okunabilirlik

10. Öneriler (öncelik sırasıyla)

- **1. BURULAŞ'tan yazılı kullanım izni** alın ya da resmi bir GTFS/açık veri kaynağına geçin; yayının ön koşulu.
- **2. Yayın için geocoding sağlayıcısı** ayarlayın (`NEXT_PUBLIC_REVERSE_GEOCODE_URL`, gerekirse `NEXT_PUBLIC_GEOCODE_URL`).
- **3. /api/route için hız sınırlama** (ör. Vercel Firewall kuralı ya da IP başına basit sayaç) ve kısa süreli yanıt önbelleği.
- **4. CI:** GitHub Actions ile her push'ta `lint + typecheck + test`; ardından kritik akışlar için birkaç Playwright testi (arama → seçim → yol tarifi).
- **5. Takvim ve ücret bakım hatırlatıcısı:** 2027 bayramlarından sonrası için bir test, tarih geçince bilerek başarısız olsun (sessiz eskime yerine).
- **6. MapShell durumunu** zamanla bir Zustand mağazasına (zaten bağımlılık) ya da useReducer'a taşıyın.
- **7. Büyük veri dosyaları** için Git LFS veya derleme sırasında indirilen sürümlü bir veri paketi düşünün.
- **8. Gerçek zamanlı veri:** BURULAŞ "Otobüsüm Nerede" araç konumları izinle kullanılabilirse tahmini saatler gerçek saatlerle düzeltilebilir.

11. Yeni geliştirici için okuma sırası

Projeyi öğrenmek için dosyaları aşağıdaki sırayla okumanız önerilir; her adım bir öncekinin üzerine kurulur.

#	Dosya	Ne öğrenirsiniz
1	README.md, package.json	Amaç, komutlar, veri kaynakları
2	data/types.ts, data/layers.ts	Place modeli, katmanlar, koordinat sırası
3	components/MapShell.tsx	Uygulama durumu ve UI parçalarının nasıl bağlandığı
4	components/map/MapView.tsx	Harita kurulumu, katman sırası, tıklamanın Place'e çevrilmesi
5	lib/search.ts → lib/place-search.ts	Arama algoritması ve tembel yüklenen indeks
6	components/ui/PlaceCombobox.tsx, RoutePanel.tsx	Arama kutusu ve yol tarifi arayüzü
7	app/api/route/route.ts → routing/refine.ts	Yol tarifi isteğinin uçtan uca akışı
8	routing/network.ts → routing/raptor.ts	Ağ modeli ve RAPTOR
9	routing/walk-graph.ts, calendar.ts, fares.ts	Yaya ağı, tarife günü, ücret
10	tests/raptor.test.ts, tests/search.test.ts	Beklenen davranışın örnekleri
11	scripts/ (osm.mjs, build-bus-dataset.mjs, fetch-walk-graph.mjs)	Verinin nereden ve nasıl üretildiği

Sık yapılan işler

İş	Nereye dokunulur
Yeni katman/kategori	data/types.ts (LayerId) → data/layers.ts → ilgili Layer bileşeni → PlaceVisuals ikonu
Yeni öne çıkan yer	scripts/fetch-pois.mjs listesi → npm run data:pois
Ücret değişikliği	lib/routing/fares.ts (FARES_AS_OF ile) + tests/fares.test.ts
Yeni yılın bayramları	lib/routing/calendar.ts (RELIGIOUS_HOLIDAYS, HOLIDAYS_KNOWN_UNTIL)
Rota parametreleri	raptor.ts sabitleri (MAX_RIDES, TRANSFER_SLACK_S, MAX_SEARCHES…) ve network.ts hızları
Otobüs verisini yenileme	README → "BURULAŞ verisini yenileme" adımları